Franka User Control

A user-friendly ROS2 control interface for the Franka Research 3 robot, providing simple Python APIs for velocity control, position control, and safety monitoring.

Features

- **Velocity Control**: Direct Cartesian velocity commands with smooth ramping
- **Position Control**: Absolute and relative position movements with configurable impedance
- **Safety Monitoring**: Built-in velocity limits, workspace boundaries, and force monitoring
- **Python API**: Simple, intuitive interface for robot control
- **Terminal UI**: Menu-driven interface for interactive control
- **Modular Design**: Easy to extend and customize for research applications

System Requirements

- **OS**: Ubuntu 22.04 (or compatible Linux distribution)
- **ROS2**: Humble or later
- **Robot**: Franka Research 3
- **Dependencies**:
  - libfranka (Franka control library)
  - Eigen3
  - ROS2 packages: rclcpp, rclpy, geometry_msgs, sensor_msgs

Installation

1. Install Dependencies

```bash
# Install ROS2 Humble (if not already installed)
# Follow: https://docs.ros.org/en/humble/Installation.html

# Install libfranka
sudo apt install ros-humble-libfranka

# Install Eigen3
sudo apt install libeigen3-dev
```

2. Create Workspace and Clone Repository

```bash
# Create workspace
mkdir -p ~/franka_user_control_ws/src
cd ~/franka_user_control_ws/src

# Clone this repository
# (Replace with actual repository URL when available)
git clone <repository-url> franka_user_control
```

3. Configure Robot IP

Edit the robot IP address in `config/robot_config.yaml`:

```yaml
robot_ip: "YOUR_ROBOT_IP_HERE"  # e.g., "192.168.1.100"
```

4. Build the Package

```bash
cd ~/franka_user_control_ws
colcon build --packages-select franka_user_control
source install/setup.bash
```

Quick Start

Testing in Simulation (No Robot Required)

```bash
# Source the workspace
source ~/franka_user_control_ws/install/setup.bash

# Launch simulation
ros2 launch franka_user_control simulation.launch.py

# In a new terminal, run tests
source ~/franka_user_control_ws/install/setup.bash
ros2 run franka_user_control test_simulation.py
```

See Simulation Guide for detailed simulation usage.

Using Real Hardware

1. Start the Control Node

```bash
# Source the workspace
source ~/franka_user_control_ws/install/setup.bash

# Launch with your robot's IP
ros2 launch franka_user_control user_control.launch.py robot_ip:=192.168.1.100
```

2. Run the Terminal UI

In a new terminal:

```bash
source ~/franka_user_control_ws/install/setup.bash
ros2 run franka_user_control terminal_ui.py
```

3. Use the Python API

```python
from franka_python_interface import FrankaInterface

# Create and connect
robot = FrankaInterface()
robot.connect()

# Velocity control
robot.set_cartesian_velocity(vx=0.05, vy=0.0, vz=0.0)  # 5 cm/s in X
# ... robot moves ...
robot.stop()

# Position control
robot.move_relative(dx=0.1, dy=0.0, dz=0.05)  # Move 10cm in X, 5cm in Z

# Disconnect
robot.disconnect()
```

Usage Examples

Example 1: Simple Velocity Control

python

```python
#!/usr/bin/env python3
import time
from franka_python_interface import FrankaInterface

robot = FrankaInterface()
robot.connect()

# Move forward for 2 seconds
robot.set_cartesian_velocity(vx=0.05, duration=2.0)
time.sleep(2.5)

# Stop
robot.stop()
robot.disconnect()
```

Example 2: Position Control with Impedance

python

```python
from franka_python_interface import FrankaInterface, ImpedanceMode

robot = FrankaInterface()
robot.connect()

# Move with soft impedance (compliant)
robot.move_relative(
    dx=0.1, dy=0.0, dz=0.0,
    max_velocity=0.1,
    impedance_mode=ImpedanceMode.SOFT,
    wait=True
)

robot.disconnect()
```

Example 3: Combined Control

python

```
# Approach with velocity, then precise positioning
robot = FrankaInterface()
robot.connect()

# Phase 1: Slow approach
robot.set_cartesian_velocity(vx=0.02, duration=3.0)
time.sleep(3.5)
robot.stop()

# Phase 2: Precise adjustment
robot.move_relative(dx=0.01, dz=-0.005, max_velocity=0.05)

robot.disconnect()
```

Testing

The package includes several test scripts:

bash

```
# Test velocity control
ros2 run franka_user_control test_velocity.py

# Test position control
ros2 run franka_user_control test_position.py

# Test combined control modes
ros2 run franka_user_control test_combined.py
```

Configuration

Safety Limits

Edit `config/safety_limits.yaml` to adjust safety parameters:

yaml

```yaml
# Velocity limits
max_translation_velocity: 0.2  # m/s
max_rotation_velocity: 1.0     # rad/s

# Workspace boundaries
workspace:
  x_min: 0.1
  x_max: 0.85
  y_min: -0.5
  y_max: 0.5
  z_min: 0.0
  z_max: 0.8
```

Impedance Modes

Edit `config/impedance_params.yaml` to customize impedance behavior:

```yaml
impedance_modes:
  stiff:
    joint_stiffness: [600.0, 600.0, 600.0, 600.0, 250.0, 150.0, 50.0]
    joint_damping: [50.0, 50.0, 50.0, 50.0, 30.0, 25.0, 15.0]
  # ... other modes
```

Architecture

```
franka_user_control/
├── C++ Bridge Node (ROS2 ↔ libfranka)
│   ├── Motion Generation (smooth trajectories)
│   ├── Velocity Control (Cartesian velocities)
│   ├── Impedance Control (compliant motion)
│   └── Safety Monitoring (limits checking)
│
└── Python Interface
    ├── FrankaInterface (main API)
    ├── Safety Monitor (Python-side checks)
    └── Terminal UI (interactive control)
```

API Reference

FrankaInterface

Main class for robot control.

Methods:

- `connect()` - Connect to robot
- `disconnect()` - Disconnect from robot
- `set_cartesian_velocity(vx, vy, vz, wx, wy, wz, duration)` - Command velocity
- `move_to_pose(target_pose, max_velocity, impedance_mode)` - Move to absolute pose
- `move_relative(dx, dy, dz, droll, dpitch, dyaw, max_velocity)` - Relative movement
- `stop()` - Stop all motion
- `emergency_stop()` - Emergency stop
- `get_current_pose()` - Get end-effector pose
- `get_joint_positions()` - Get joint angles
- `set_velocity_limits(max_linear, max_angular)` - Update safety limits

Control Modes

- `ControlMode.IDLE` - No active control
- `ControlMode.VELOCITY` - Velocity control mode
- `ControlMode.POSITION` - Position control mode

Impedance Modes

- `ImpedanceMode.STIFF` - High stiffness (precise positioning)
- `ImpedanceMode.MEDIUM` - Balanced (general use)
- `ImpedanceMode.SOFT` - Low stiffness (compliant interaction)
- `ImpedanceMode.VERY_SOFT` - Very low stiffness (contact tasks)

Safety

Important Safety Considerations:

1. **Always** have the emergency stop button accessible
2. **Start with conservative limits** when testing new code
3. **Monitor the robot** during all automated motions
4. **Verify workspace limits** before operation
5. **Test new trajectories** at low velocities first

The system includes multiple safety layers:

- Python-side safety checks (pre-validation)
- C++ safety monitoring (real-time)
- Franka built-in safety reflexes (hardware)

Troubleshooting

Robot won't connect

- Verify robot IP address in config
- Check network connection: `ping <robot-ip>`
- Ensure robot is unlocked in Franka Desk
- Check firewall settings

Safety reflex triggered

- Reduce velocity/acceleration limits
- Check collision behavior settings
- Ensure smooth velocity ramping
- Verify workspace boundaries

Motion feels jerky

- Increase ramp time in velocity controller
- Check control loop frequency
- Verify impedance settings

Development

Adding New Features

The modular design makes it easy to extend:

1. **Add new control mode**: Implement in C++ controller, expose via service
2. **Custom safety checks**: Extend SafetyMonitor class
3. **New UI**: Use FrankaInterface as backend

Code Structure

- `src/` - C++ implementation

- `include/` - C++ headers
- `franka_python_interface/` - Python API
- `scripts/` - Executable scripts
- `config/` - Configuration files
- `launch/` - Launch files

Contributing

Contributions are welcome! Please:

1. Fork the repository
2. Create a feature branch
3. Add tests for new features
4. Submit a pull request

License

Apache License 2.0

Authors

- Your Name

Acknowledgments

- Based on Franka Robotics' libfranka examples
- Uses libfranka control library
- Built with ROS2

Support

For issues and questions:

- GitHub Issues: (link to issues page)
- Documentation: (link to docs)
- Franka Robotics Support: https://franka.de/support

Citation

If you use this software in your research, please cite:

bibtex

```
@software{franka_user_control,
  title = {Franka User Control: A User-Friendly ROS2 Interface for Franka Research 3},
  author = {Your Name},
  year = {2024},
  url = {https://github.com/...}
}
```